

Control de versiones (git)

Álvaro González Sotillo

28 de diciembre de 2025

Índice

1. Introducción	1
2. Trabajo colaborativo	1
3. GIT	2
4. Comandos de Git	4
5. Git desde IntelliJ	7
6. Si todo se complica...	8
7. Referencias	9

1. Introducción

Un desarrollo suele ser un trabajo colaborativo entre varias personas. Por ello, hay que resolver varios problemas:

- Cómo dividir el trabajo en varias partes (módulos)
- Cómo integrar las partes para conseguir el resultado final.
- Cómo comunicar instrucciones para:
 - Instalar un entorno de desarrollo (cuando llegan nuevos desarrolladores)
 - Instalar la aplicación en el servidor de producción
 - Utilizar los módulos de otras personas
 - Traspasar el conocimiento (cuando un desarrollador deja el equipo)

2. Trabajo colaborativo

2.1. Integración de código

- Cada miembro del equipo realiza su parte del trabajo
- Al terminar
 - ¿Cómo se integran todas las partes?
- Si hay errores o cambios
 - ¿Cómo se actualizan solo los nuevos cambios?
- Si hay más de una versión
 - ¿Cómo se gestionan los conjuntos de módulos compatibles con cierta versión?
 - ¿Cómo se recupera una versión antigua?
 - ¿Cómo se fusionan dos versiones?

2.2. Control de versiones

- Un control de versiones (de código) mantiene una base de datos con todos los cambios producidos sobre el código fuente y otros ficheros necesarios para generar una aplicación
- Pueden ser
 - Centralizados/distribuidos
 - Basados en ficheros/Basados en *changesets*
 - Bloqueantes/No bloqueantes
- Ejemplos
 - GIT
 - Subversion
 - Mercurial

Control de versiones en Wikipedia

2.3. Conceptos de control de versiones (I)

- **Repositorio:** Base de datos con las sucesivas versiones
- **Copia de trabajo:** Ficheros sobre los que trabaja el programador
- **Commit:** Confirmación de una nueva versión (de la copia de trabajo al repositorio)
- **Checkout:** (Re)generación de la copia de trabajo a partir de la información del repositorio
- **Changeset:** Conjunto de cambios en el repositorio, considerado atómico

2.4. Conceptos de control de versiones (II)

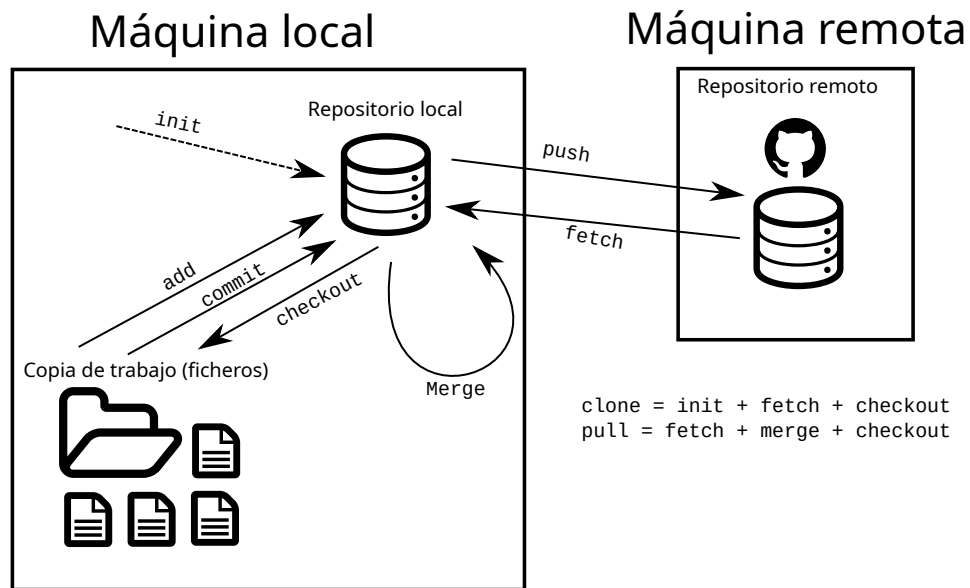
- **Rama (Branch):** Versión paralela (ni anterior, ni posterior)
- **Rama master o trunk:** Rama principal del desarrollo
- **Fusión (Merge):** Acumular cambios en una misma versión
 - Tras un *checkout*, mezclando la copia de trabajo con la versión del repositorio
 - Entre diferentes branches del repositorio
- **Comentario:** Cada commit debería documentar sus efectos y motivaciones.

3. GIT

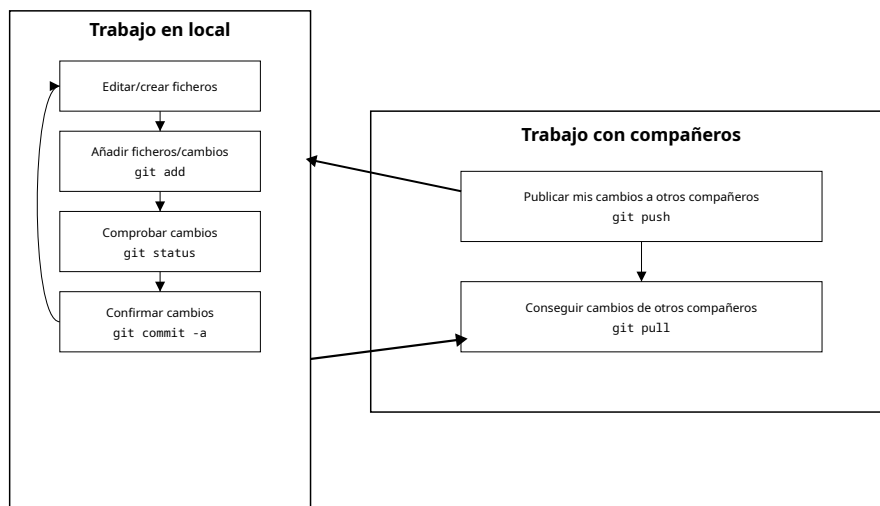
- Gratis, de código abierto
- Muy popular, en múltiples plataformas
- Distribuido
- Basado en *changesets*
- No bloqueante

3.1. GIT como cliente-servidor

- Git tiene una arquitectura **peer-to-peer**
 - Todos los repositorios son igual de "*importantes*"
- Por convenio, suele haber un repositorio "*central*"
 - Como servicio: **github**, **gitlab**, **codeberg**...
 - Autoalojado: **gitea**, **forjengo**
 - **O solo con Git y linux**



3.2. Flujo de trabajo básico



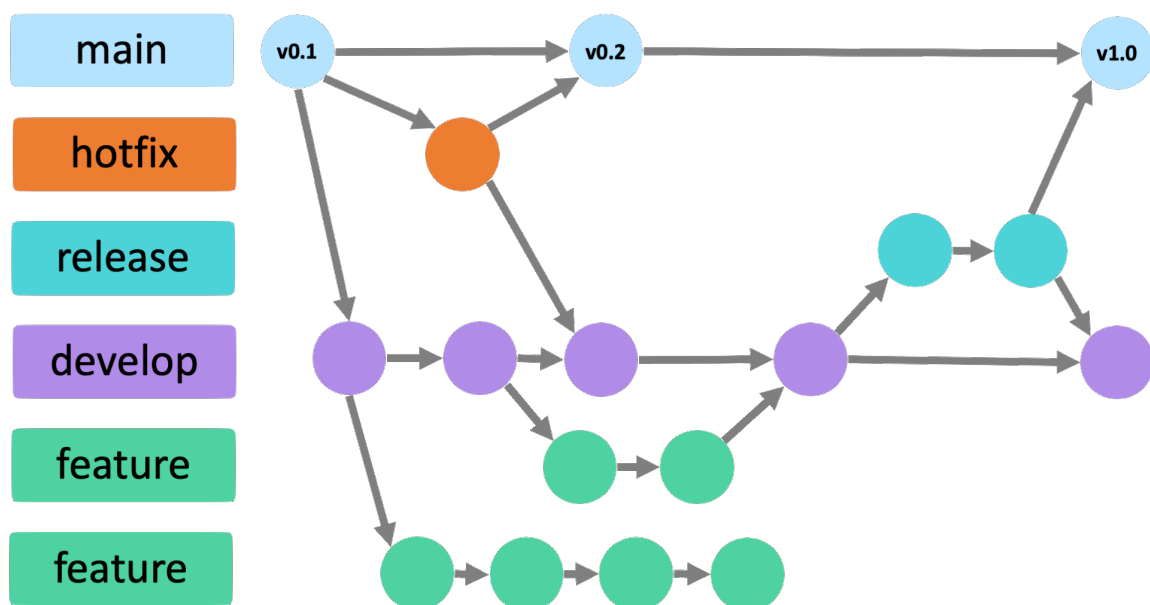
3.3. Ramas

- Cada *commit* genera una nueva versión del código
- Generalmente, una versión "*desciende*" de otra versión
 - En un *merge*, se puede "*descender*" de más de una versión

- Las versiones se identifican por un *hash*
- Una **rama** (branch) es un nombre que se da a un *hash*, y que va avanzando al hacer nuevas versiones
- Un tag es como un branch, pero no avanza al hacer nuevas versiones

3.3.1. Ramas típicas

- master o main: Versiones oficiales del código
 - Se garantiza que funcionan
 - Se entregan al usuario final
- develop: Trabajo para las siguientes versiones oficiales
 - Se acaban fusionando (*merge*) en master
- feature: Trabajo para una sola funcionalidad
 - Se acaban fusionando en develop
 - Pueden ser personales: solo las utiliza un desarrollador
- hotfix: Un arreglo urgente en master



4. Comandos de Git

4.1. `init`

- Crea un nuevo repositorio en el directorio
- Inicialmente vacío
- Se almacena en el directorio oculto `.git`

```
alvaro@debian8-64-alvarogonzalez:~/aplicacion-web$ git init
Initialized emp
```

4.2. clone

- Crea un nuevo repositorio, copia de uno ya existente
- El repositorio puede ser local o remoto, dependiendo de la **URL**
- La copia de trabajo será la de la rama master

```
alvaro@debian8-64-alvarogonzalez:~/aplicacion-web$ git clone \
https://github.com/alvarogonzalezsotillo/aplicacion-php.git
Cloning into 'aplicacion-php' ...
remote: Counting objects: 50, done.
remote: Compressing objects: 100% (35
```

Comando *git clone*

4.3. add

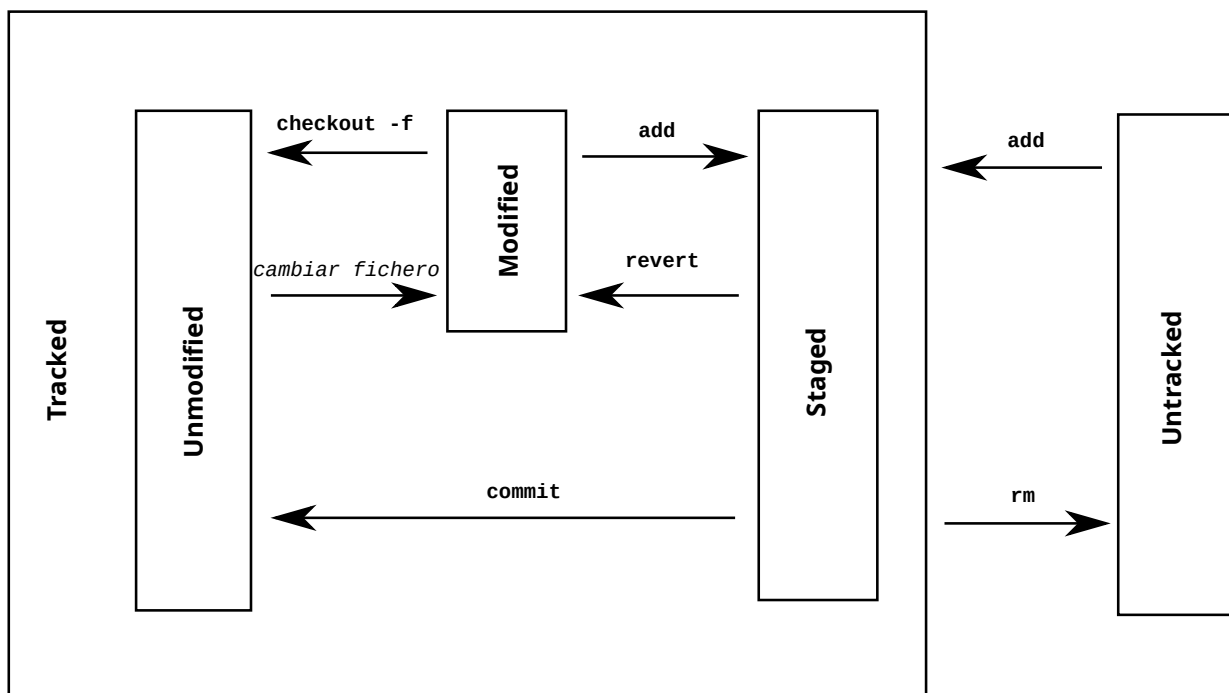
- Git no incluye todos los ficheros de la copia de trabajo en el repositorio
- Si se desea un fichero en el repositorio, se debe incluir con add
- Cada cambio posterior debe indicarse también con add
- Se puede quitar con rm (las versiones antiguas siguen en el repositorio)

Comando *git add*

4.4. status

- Imprime un resumen del estado de los ficheros de la copia de trabajo
 - **Untracked:** No guardados en el repositorio. Git ignora el fichero
 - **Unmodified:** Igual que en la rama activa del repositorio
 - **Modified:** Con cambios respecto al repositorio, pero que no está previsto guardar

Comando *git status*



4.5. diff

Muestra los cambios de los ficheros en estado **modified**

- `@@ -lineO,nlinesO +lineD,nlinesD @@`
 - Se muestra a partir de `lineO` del fichero antiguo y `lineD` del fichero actual
 - Se muestran `nlinesO` líneas del fichero antiguo y `nlinesD` líneas del fichero actual
- `-` Línea borrada
- `+` Línea añadida

Comando *git diff*

4.6. commit

```
[master 906fa20] Cambiados varios permisos de ejecución
5 files changed, 411 insertions(+), 2 deletions(-)
create mode 100644 09/estados-fichero-git.svg
mode change 100755 => 100644 borrar-temporales-latex.sh
mode change 100755 => 100644 generar-pdf.sh
mode change 100755 => 100644 generar-pdfs.sh
mode change 100755 => 100644 publicar-pdfs.sh
```

Comando *git commit*

4.7. push

- Sube las versiones del repositorio local a un repositorio remoto
- El repositorio remoto puede establecerse:
 - con `git clone` (en este caso se denomina **origin**)
 - con `git remote add`
- El repositorio debe tener todas las versiones del repositorio remoto
 - En otro caso, deberá realizarse un `git pull` previo

Comando *git push* Comando *git remote*

```
,numbers=none] alvaro@debian8-64-alvarogonzalez: /aplicacion-webgitpushPasswordfor'https : //alvarogonzalezsotillo@b
Tohttps : //alvarogonzalezsotillo@bitbucket.org/alvarogonzalezsotillo/aplicacion - web.git![rejected]master- >
master(fetch first)error : failedtopushsomerefsto'https : //alvarogonzalezsotillo@bitbucket.org/alvarogonzalezsotillo/apli
web.git'hint : Updateswererejectedbecausetheremotecontainsworkthatyou dohint : nothavelocally.This isusuallycausedbyanot
tothesameref.Youmaywanttofirstintegrate theremotechangeshint : (e.g.,' gitpull...')beforepushingagain.hint : Seethe'Notea
forwards'in' gitpush --help'fordetails.
```

4.8. checkout

- Extrae versiones del repositorio a la copia de trabajo
 - **Un fichero:** `git checkout <fichero>`
 - **Una versión:** `git checkout <hashdeversión>`
 - **Un branch:** `git checkout <branch>`
- También crea nuevas ramas, con `git checkout -b <nombrerama>`
- Si coinciden una versión/nombre de fichero
 - Utilizar **Referencias a objetos en Git**
 - Es **mejor** evitar que coincidan

Comando *git checkout* Comando *git branch*

4.9. merge

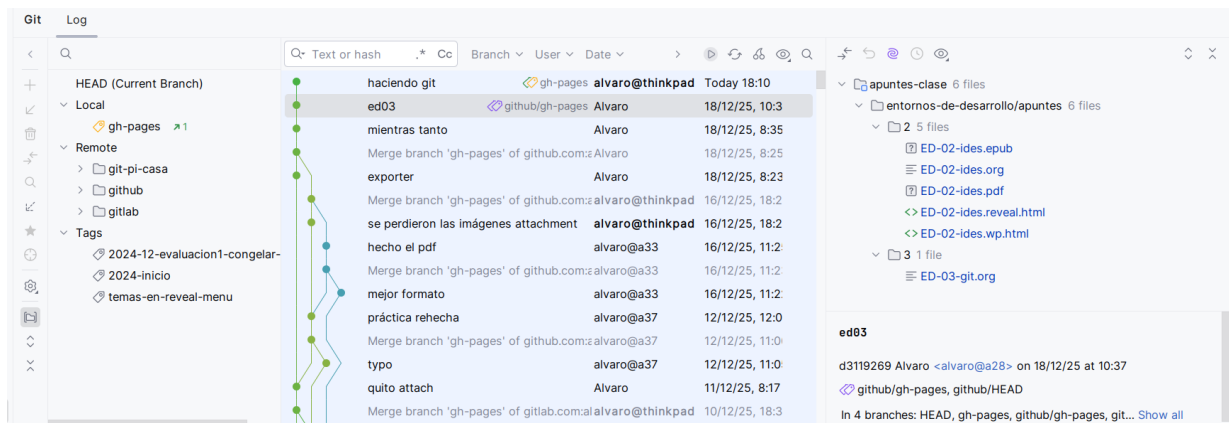
- Fusiona la rama actual con otra rama
- En el caso de que haya conflictos:
 - Los ficheros se quedan con **marcas de conflictos**
 - Para aceptar el fichero completo de la otra rama: `git checkout --theirs path/file`
 - Para aceptar el fichero completo de la rama actual: `git checkout --ours path/file`
 - Para casos más complicados, editar los ficheros y confirmarlos con `git add`
- Recomendación: `git mergetool`, o usar el IDE

Comando `git merge`: Fusión de dos ramas

4.10. Otros comandos

- Comando `git blame`: Lista un fichero con el autor de cada línea
- Comando `git log`: Historia de cambios de un repositorio (parecido a `gitk` en modo texto)
- Comando `git gui`: Interfaz gráfica básica para utilizar `git`

5. Git desde IntelliJ



6. Si todo se complica...

THIS IS GIT. IT TRACKS COLLABORATIVE WORK ON PROJECTS THROUGH A BEAUTIFUL DISTRIBUTED GRAPH THEORY TREE MODEL.

COOL. HOW DO WE USE IT?

NO IDEA. JUST MEMORIZE THESE SHELL COMMANDS AND TYPE THEM TO SYNC UP. IF YOU GET ERRORS, SAVE YOUR WORK ELSEWHERE, DELETE THE PROJECT, AND DOWNLOAD A FRESH COPY.



7. Referencias

- Formatos:
 - [Transparencias](#)
 - [PDF](#)
 - [Página web](#)
 - [EPUB](#)
- Creado con:
 - [Emacs](#)
 - [org-re-reveal](#)
 - [Latex](#)
- Alojado en [Github](#)